DIGITAL VLSI DESIGN · COURSE PROJECT · LEARNING GUIDE

Leakage, Learned

An intuitive, end-to-end guide to estimating CMOS leakage under process, voltage and temperature variation with machine learning: from one transistor to a 204-transistor adder.



log I vs V : the exponential that the whole project is about

Team Leaky Lima (Team 1)

VDD 1.0 V nominal (0.9 – 1.1 V)

Simulator ngspice-44.2

Model PTM 16 nm High-Performance

Test circuit 3-bit carry-lookahead adder

Built 01 November 2026

Contents

Read it in order the first time. Each chapter starts with the intuition, then the theory, then what we actually did (with the file names), and ends with viva questions.

0 The project on one page
What we built, the headline numbers, how the pieces connect

1 Why leakage, why PVT, why ML
The problem in plain words

2 Leakage physics, intuitively
Sub-threshold, tunnelling, junctions; vth0, toxe, temperature, DIBL, body effect

3 Part A1: Making the data with SPICE
The four cases, sampling, the speed trick, a silent simulator bug, the data check

4 Machine learning from zero
Regression, splits, overfitting, features, log targets, the four model families, metrics

5 The key idea: a transistor is six bipoles
Why direct prediction fails, passivity, log G, symmetry, the model architecture

6 Part A2: Training and choosing the models
Model comparison, chosen models, distribution shift and the rail fix, learning curve

7 Part B: From transistors to gates
Leakage definition, Tellegen's theorem, the stack effect, gate results

8 Part C: The 3-bit carry-lookahead adder
Building it from gates, hierarchical estimation, splitting the error into its causes

9 Error analysis, limitations, improvements
Where it is wrong, why, and what we would do next

10 Viva preparation
Questions you are likely to get, with short answers

A Appendix: files, how to re-run, glossary

0 The project on one page

We teach a computer how *one* transistor leaks under any process, voltage and temperature condition. Then we predict the leakage of whole gates and a whole adder by adding up transistors, with no circuit-level SPICE run needed.

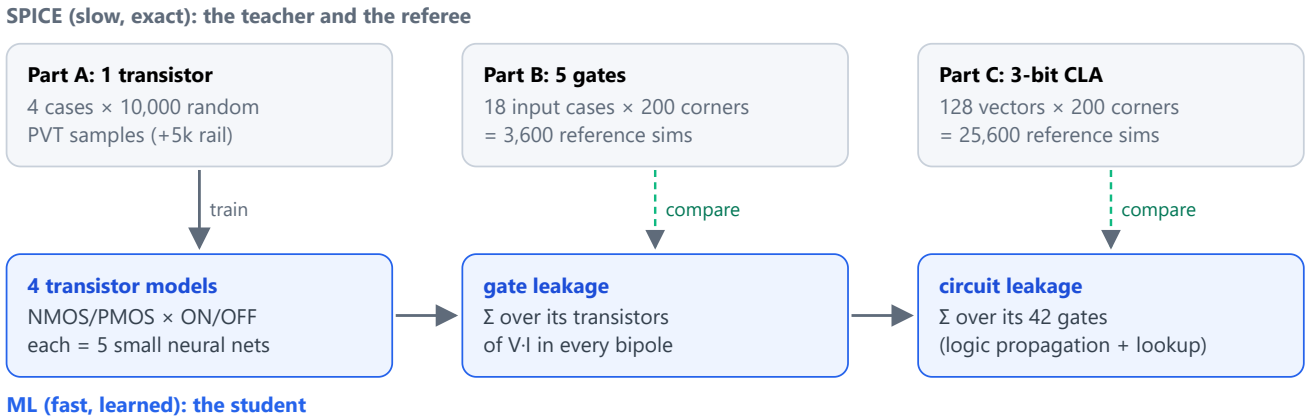


Figure 0.1. The whole project. SPICE does two jobs: it generates training data (Part A) and it is the reference we check against (Parts B, C). The ML side never sees any gate or circuit simulation while it is trained.

0.57 %
median transistor error (test set, average of 16 current models)

1.40 %
median gate error (3,600 gate sims)

2.85 %
median adder error (25,600 circuit sims)

96.13 %
mean accuracy on the adder (100 % − mean |error|)

The five ideas that made it work

1. **Fast, verified SPICE.** One ngspice process loops over thousands of samples (40,000 sims in about a minute). It is checked against standalone netlists, and that check caught a silent simulator bug.
2. **Six bipoles, not four terminals.** Terminal currents are sums of mechanisms that cancel each other. Each mechanism on its own is a smooth, one-signed "non-linear resistor", which is easy to learn as *log G*.
3. **Smooth learners for smooth physics.** Tree models give staircases. A neural network trained with L-BFGS fits exponential laws to well under 1 %.
4. **Train where you deploy, and debug with physics.** Gates keep their transistors at the supply rails, a region uniform sampling almost never visits. Adding rail-focused samples, and two physics features found by a per-transistor "oracle" check (effective width and gate-voltage offset), cut the gate error by more than half.
5. **Leakage = power ÷ VDD.** Defined this way, gate leakages add up to circuit leakage, and every ML term is $G \cdot V^2 \geq 0$, which is robust to small errors.

1 Why leakage, why PVT, why ML

A switched-off chip should draw no current. Real transistors are leaky taps, and a billion leaky taps add up to a flood.

> **INTUITION**
> Think of each transistor as a water tap. Ideally a closed tap (OFF transistor) lets through no water. At 16 nm the tap washer is a few atoms thick and the tap is so short that it never fully seals: there is always a drip. A phone chip has billions of taps, so even a drip of 100 nA each becomes amperes of wasted current, and that drains your battery while the screen is off.

Why the drip changes so much: PVT

The size of the drip is not fixed. It depends on:

- **Process (P).** No two transistors come out of the factory identical. The threshold voltage, oxide thickness, doping, mobility and junction depth all vary by a few percent. In this project each varies by ±10 % (`vth0, toxe, ndep, u0, xj`).
- **Voltage (V).** The supply is never exactly 1.0 V; we cover 0.9-1.1 V. A higher drain voltage also lowers the transistor's threshold (DIBL, Chapter 2), so leakage rises faster than the voltage.
- **Temperature (T).** −40 to 125 °C. Hotter electrons find it easier to cross the barrier.

Leakage depends *exponentially* on several of these, so the same gate can leak 10× more at one corner than at another. A designer needs leakage at thousands of corners, for thousands of gates, for every input pattern. That is a lot of SPICE.

Why machine learning

SPICE solves the full non-linear circuit equations: exact, but slow for big circuits and many corners. Our idea is to **learn the transistor once**, from SPICE, as a fast function. After that, any gate or circuit is just a sum of transistor evaluations. SPICE is still used, but only to (a) teach the transistor models and (b) check our answers.

> **IN OUR PROJECT**
> Our assignment (Team 1, "Leaky Lima"): PTM **16 nm high-performance** model card `models/16nm_HP.pm`, nominal VDD **1.0 V**, test circuit a **3-bit carry-lookahead adder**. All of this lives in one file, `config.py`, so every script uses the same values. One detail worth remembering: PTM's own nominal VDD for this card is 0.7 V. Running it at 1.0 V makes leakage unusually large (about 0.1-3 µA per gate), which is a good stress test.

VIVA CHECK

Why not just simulate everything in SPICE? For our adder SPICE needed about 1,200 s for the 25,600 operating points on an otherwise idle 16-thread laptop with 12 parallel ngspice workers (the final rerun took 7628 s because it shared the CPU with model training), against 8.24 s for the hierarchical ML. The gap widens with circuit size, because SPICE cost grows super-linearly with the number of nodes, while the ML cost is a lookup and a sum per gate.

2 Leakage physics, intuitively

Three ways current sneaks through a "closed" transistor. Every one of them is exponential in something, and that single fact shapes every later decision.

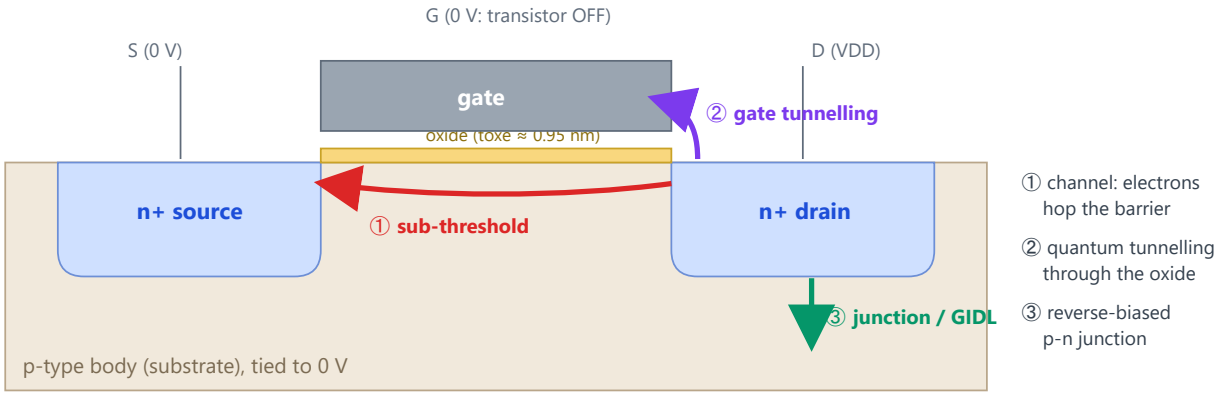


Figure 2.1. An OFF NMOS (gate at 0 V, drain at VDD) and its three leakage paths. ① dominates almost everywhere in our circuits, ② is the next biggest, and ③ is tiny in this model card (about 10^{-14} A).

① Sub-threshold leakage: the big one

Between source and channel sits an energy hill, the "barrier". The gate voltage controls its height. Below threshold the hill is tall, but electrons have a spread of thermal energies (a Boltzmann distribution), so a few always have enough energy to get over. The fraction that make it falls off exponentially with the barrier height:

$$I_{sub} \approx I_0 \cdot (W/L) \cdot \exp[\,(V_{GS} - V_{th}) / (n \cdot V_T)\,] \cdot [\,1 - \exp(-V_{DS}/V_T)\,], \quad V_T = kT/q \approx 26\ \text{mV}$$

Read it term by term. **W/L**: a wider channel means more parallel paths. **$\exp[(V_{GS}-V_{th})/nV_T]$**: every 60-90 mV of gate voltage changes the current by 10×. **$[1 - \exp(-V_{DS}/V_T)]$**: once V_{DS} is a few V_T this is ≈ 1, and at V_{DS} = 0 it is 0 (no push, no flow).

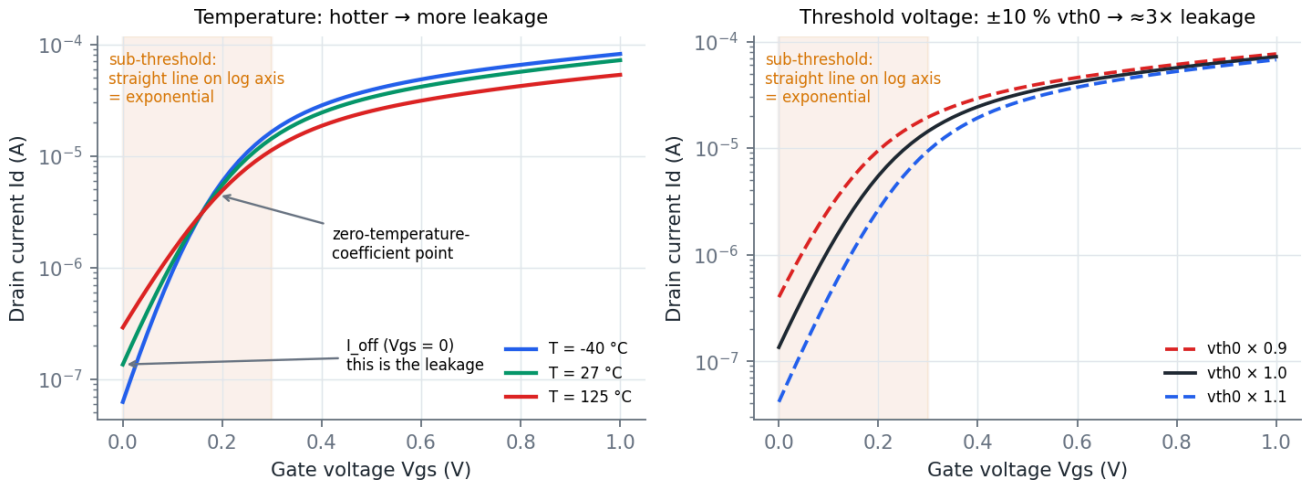


Figure 2.2. Real SPICE I-V curves of our 16 nm NMOS (W = 32 nm, V_{DS} = 1 V). On a log axis the sub-threshold region is a straight line, which is what "exponential" looks like. The leakage is the value at V_{GS} = 0: about 135 nA nominal. Left: hotter means more leakage, but only below the *zero-temperature-coefficient point*; above it the drop in mobility wins. Right: ±10 % on vth0 moves the leakage by about 3× either way.

Four effects that move V_{th}, and so the leakage

Effect	What happens	Effect on leakage	Where you see it in this project
vth0 variation	The built-in threshold varies with process	$\exp(-\Delta V_{th}/nV_T)$: +10 % vth0 (48 mV) → ×0.31	Trend plot (Ch. 3), corner error analysis
Temperature	V_T grows and V_{th} drops as T rises	25 → 125 °C: ×2.2 here (weaker than in older nodes)	Trend plot
DIBL (drain-induced barrier lowering)	In a 16 nm channel the drain is so close that its voltage pulls the barrier down	Higher V_{DS} → lower V_{th} → more leakage	Why 1 V (instead of 0.7 V) gives huge leakage
Body effect	Source raised above body (V_{SB} > 0) → V_{th} rises	Raised source → much less leakage	**The stack effect** (Ch. 7)

② Gate (oxide) tunnelling

The gate oxide here is under one nanometre, just a few atomic layers. Quantum mechanics lets electrons *tunnel* straight through a thin barrier, with a probability that falls exponentially with thickness. So `toxe` is the knob: in our data, +10 % toxe gives ×0.23 gate current. Tunnelling happens wherever there is voltage across the oxide: gate↔source, gate↔drain (the overlap regions) and gate↔channel/body.

③ Junction and GIDL leakage

The drain and source are n-type islands in a p-type body, i.e. p-n diodes. In normal operation they are reverse-biased and leak a tiny saturation current. GIDL (gate-induced drain leakage) adds band-to-band tunnelling where the gate overlaps the drain. In ON devices, impact ionisation (`isub`) also pushes some current into the body. In this model card all of these are around 10^{-14} A, millions of times smaller than ①.

GOING DEEPER: A QUIRK OF OUR MODEL CARD

The drain junction current is stuck at almost exactly 1.000×10^{-14} A and the source junction at about 10^{-18} A, whatever the voltage or temperature. A real diode's leakage should grow strongly with temperature. This happens because our netlists do not specify diffusion areas (AS/AD), so BSIM4 falls back to geometry defaults (`geomod = 1`). It doesn't affect gate leakage (too small to matter), but it matters for modelling: it is why we test symmetry instead of assuming it (Chapter 5).

VIVA CHECK

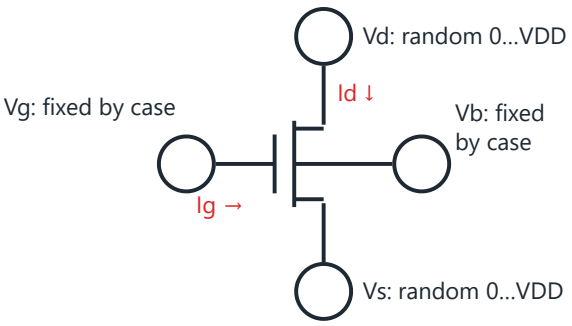
Which leakage mechanism dominates in your circuits, and how do you know? Sub-threshold. In our data the OFF-channel current at V_{DS} = VDD is about 10^{-7} A, while gate tunnelling is about 10^{-11} A and junctions about 10^{-14} A. Our model also predicts each mechanism separately, so we can read the split directly.

Why does leakage fall so much when vth0 rises 10 %? Because it is exponential in $V_{th}/(nV_T)$: 48 mV / (1.5 × 26 mV) ≈ 1.2, and $e^{-1.2} \approx 0.30$. Our SPICE sweep gives 0.31.

3 Part A1: Making the data with SPICE

To learn how a transistor leaks we need examples: inputs (voltages, temperature, process) paired with the currents SPICE computes for them. 40,000 of them.

One sample = one single-transistor simulation



Every terminal has its own source, so SPICE reports each current

Case	V_g	V_b	Means
NMOS ON	VDD	0	NMOS with input = 1
NMOS OFF	0	0	NMOS with input = 0
PMOS ON	0	VDD	PMOS with input = 0
PMOS OFF	VDD	VDD	PMOS with input = 1

Why four cases are enough: in static CMOS every gate terminal sits at a logic level and every body is tied to a rail. Only drain and source depend on the circuit, so those are the random inputs.

Drawn independently, uniformly, for every sample (VDD first, since V_d and V_s depend on it): VDD ∈ [0.9, 1.1] V · V_d, V_s ∈ [0, VDD] · W ∈ [16, 128] nm · T ∈ [−40, 125] °C · vth0, toxe, ndep, u0, xj ∈ nominal ± 10 %. Output: `id, ig, is, ib`, positive when the current flows *into* the transistor, so Kirchhoff's law reads $I_d+I_g+I_s+I_b = 0$.

The speed trick: one simulator, thousands of samples

Starting ngspice costs about 0.1 s. Starting it 40,000 times would waste over an hour. Instead each of 12 parallel workers writes **one** netlist that loops over its share of samples. Inside the loop, `altermod` changes the process parameters, `alter` changes the width and the voltages, and `option temp` changes the temperature. Then `op` solves, and `wrdata` appends one line of results.

```
altermod nmos vth0=4.41e-01      ← process parameters, ONE PER LINE (see below)
altermod nmos toxe=8.80e-10
 ...
alter m1 w=7.85e-08              ← width
alter vd dc=3.98e-01             ← terminal voltages (vg, vs, vb likewise)
option temp=-36.7                ← temperature
op                               ← solve the DC operating point
let idx = 0
wrdata out.txt idx i(Vd) i(Vg) i(Vs) i(Vb) @m1[igs]  ...    ← append results
```

Result: **40,000 simulations in about 70 seconds, zero convergence failures**. The `idx` column guards against a subtle failure: if one sample failed to converge, its row would be missing rather than silently shifting every later result by one.

PITFALL WE HIT: A SILENT SIMULATOR BUG

A fast method is only trustworthy if it gives the *same* answers as the slow, obviously-correct one. So the generator re-simulates a few samples with a **fresh netlist whose model card is edited as text**, and compares. On the first run they disagreed by up to **1200 %**. Testing one parameter at a time showed each worked alone, but `altermod nmos vth0=… toxe=… ndep=…` **silently applied only the first parameter** in ngspice-44.2. Four of our five process parameters were doing nothing. Fix: one `altermod` per line. Afterwards the two methods agree to 2×10^{-6}, the print precision.

Lesson: always validate a fast path against a slow trusted path. Without that check the whole project would have looked fine and been wrong.

Data check (deliverable 3)

Case	median |ΣI| / max|I|	95th pct	worst	Verdict
nmos_on	9.8e-10	4.3e-08	1.9e-03	✓ balanced
nmos_off	8.9e-10	3.7e-09	8.3e-09	✓ balanced
pmos_on	8.6e-10	1.5e-08	4.3e-03	✓ balanced
pmos_off	2.1e-04	2.0e-02	1.4e-01	✓ (noise floor, see text)

Kirchhoff holds to about one part in a billion. PMOS-OFF only *looks* worse because its currents are tiny: its gate current is around 10^{-15} A and comes out as values like 8.882×10^{-16} and 5.551×10^{-16}, which are exact multiples of double-precision rounding error (2.2×10^{-16}). Those numbers are noise. So from here on **percentage errors are measured only for currents above 10^{-14} A**, and we report what fraction of samples that keeps.

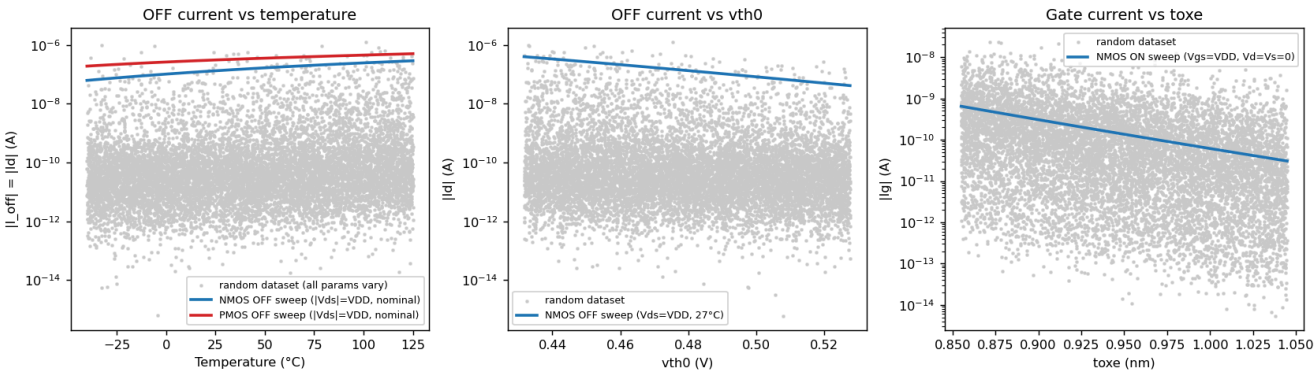


Figure 3.1. The three trend plots. With every parameter random at once a raw scatter (grey) is a cloud, so each panel overlays a controlled SPICE sweep (coloured): only that one parameter moves, everything else is nominal. That shows cause and effect clearly. Notice the cloud mostly sits *below* the line: random samples rarely put the source exactly at 0 V and the drain at VDD, which is the condition real gates operate in. Remember this; it comes back in Chapter 6.

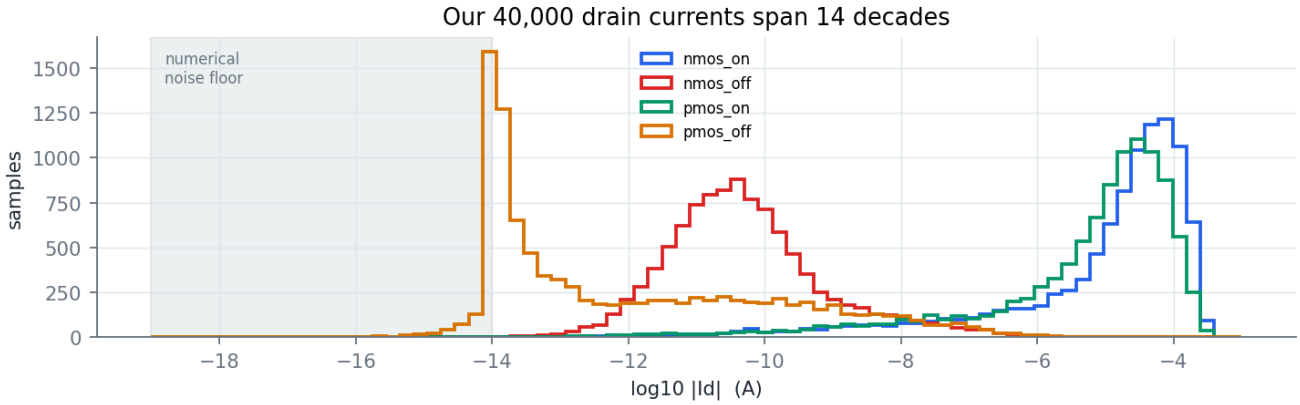


Figure 3.2. The currents span 14 orders of magnitude. This is why every model in this project works on a *logarithmic* scale (Chapter 4).

> **IN OUR PROJECT**
>
> `01_generate_transistor_data.py` (sampling, simulation, two self-checks) · `leakml/spice.py` (the batching engine) · `02_data_check.py` (Kirchhoff and trends) · outputs `data/<case>.csv` (the 10,000-row deliverables), `netlists/single_<case>.cir`, `results/kirchhoff_check.csv`.

4 Machine learning from zero

ML here means one thing: fit a function to examples so that it is also right on examples it has never seen.

Supervised regression in one paragraph

We have pairs ($\mathbf{x}$, y): x = the inputs (VDD, T, W, V_d, V_s, five process parameters), y = a current from SPICE. A *model* is a function f with adjustable knobs (parameters). *Training* turns the knobs to make f(x) close to y on the training examples, usually by minimising the mean squared error. The real test is *generalisation*: is f right on new x?

Train / validation / test: three jobs, three datasets



Figure 4.1. Why three sets? If you choose between models using the test set, you have tuned to it and its score becomes optimistic. The test set must be touched exactly once. We split with a fixed seed (42) so it is reproducible.

> **INTUITION: OVERFITTING**
>
> A student who memorises last year's answers scores 100 % on last year's paper and fails this year's. A model with too much freedom can memorise its training points, including their noise, and wiggle wildly between them. The validation set is "this year's paper": it tells us whether the model learnt the rule or just memorised.

Two choices that matter more than the model

1. The target scale. Currents span 14 decades. Fitting raw amperes with squared error, a 10^{-5} A current dominates completely and a 10^{-12} A current is invisible. Fitting **log(current)** instead means an error of 0.01 in the log is a 1 % error *for every current*. Relative accuracy is exactly what the brief asks for.

2. The features. A model only knows what it is shown. Physics tells us currents go like $\exp(V/V_T)$, so we feed V/V_T directly (where $V_T = kT/q$) instead of hoping the model discovers the combination. Likewise $\log|V|$, the log of the *effective* width, and process parameters normalised to their nominal value (so they are all $\approx 1 \pm 0.1$). Chapter 6 shows how two of these features were added only after the errors pointed to them.

The four model families (the brief asks us to compare different types)

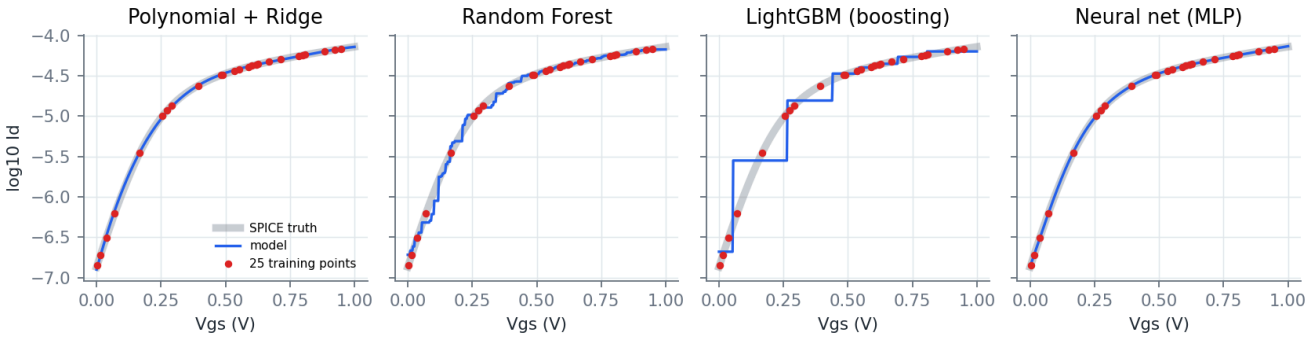


Figure 4.2. The four families fitting the same real SPICE curve (log I_d vs V_{GS}) from 25 points. Trees (Random Forest, LightGBM) predict *constants in boxes*, so they draw staircases. The polynomial and the neural net are *smooth*. Physics laws are smooth, so smooth learners have a built-in advantage. In 12 dimensions, though, a polynomial runs out of flexibility (Chapter 6).

Family	How it works	Main hyper-parameters we tried
Polynomial + Ridge	Creates every product of the inputs up to degree 3 or 4 (e.g. T·vth0, V_d^2·W) and fits a weighted sum. "Ridge" adds a penalty on large weights so it does not overfit; the penalty strength is chosen by cross-validation.	degree 3, 4; ridge α from 10^{-8} to 1
Random Forest	Grows hundreds of decision trees, each on a random resample of the data, and averages them. Each tree asks yes/no questions ("is V_d > 0.4?") and predicts the average of the training points in its final box.	300 trees, 60 % features per split
LightGBM (gradient boosting)	Builds small trees one at a time; each new tree fits the *mistakes* of all previous trees. Usually the best on spreadsheet-like data.	4,000 trees, learning rate 0.02, 31 or 63 leaves
MLP neural network	Layers of neurons, each computing tanh(w·x + b); stacking layers gives a smooth function that can bend in any way needed. We use 3 hidden layers of 64 or 128 neurons.	(64,64,64) and (128,128,128), tanh, L-BFGS

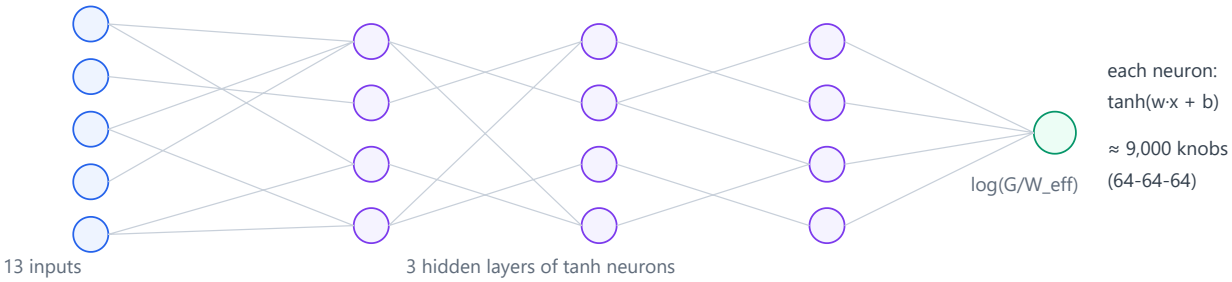


Figure 4.3. The neural network we use (a "multi-layer perceptron"). Inputs and output are standardised (mean 0, std 1) before training, which keeps the tanh neurons in their useful range.

GOING DEEPER: WHY L-BFGS AND NOT ADAM?

Training means walking downhill on an error landscape. **Adam** (the usual deep-learning choice) takes many small, noisy steps; it is built for millions of noisy examples. Our data is small (≈10k) and *noise-free* (SPICE is deterministic). **L-BFGS** uses the whole dataset each step and estimates the landscape's *curvature*, so it can take confident, well-aimed steps (a quasi-Newton method). In our early experiments, switching from Adam to L-BFGS cut the median error from about 2 % to about 1 % with nothing else changed.

How we score a model

- **Relative error** = |ML − SPICE| / |SPICE| per sample. We report the **median** (the typical case) and the **95th percentile, p95** (95 % of predictions are better than this; it measures the bad tail).
- **Bias** = median of the *signed* error: does the model systematically over- or under-predict?
- **R^2** = 1 − (unexplained variance / total variance), computed on a signed-log scale so each decade counts equally. 1.0 is perfect. It is a coarse measure here: every serious model scores above 0.99, which is why we rely on median and p95.
- **Mean accuracy** = 100 % − mean |relative error| (the "99 % mean accuracy" target in the brief).

5 The key idea: a transistor is six bipoles

Our first attempt, predicting I_d, I_g, I_s, I_b directly, stalled at about 3-20 % error. The fix came from looking inside the transistor, exactly as slide 1 of the course material suggests.

Why predicting terminal currents directly is hard

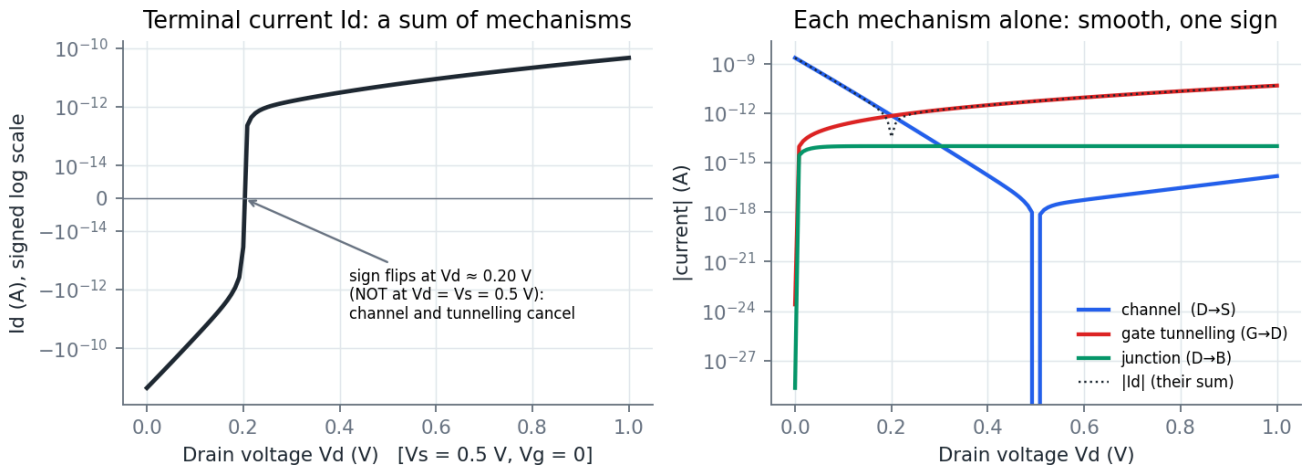


Figure 5.1. Real SPICE data, NMOS OFF, V_s = 0.5 V, sweeping V_d. Left: the drain current changes sign with a near-vertical cliff at V_d ≈ 0.2 V, a point that is not special at all for the physics. Right: the cliff is just two smooth mechanisms (the channel, flowing out of the drain, and gate tunnelling, flowing in) cancelling each other. Each mechanism alone is smooth, has one sign, and is easy on a log scale.

On a log scale a terminal current that crosses zero plunges to $-\infty$ and comes back with the other sign. No smooth model can follow that, and the location of the cliff moves around with every PVT parameter. The sum is hard; the parts are easy. **So we learn the parts.**

Six bipoles

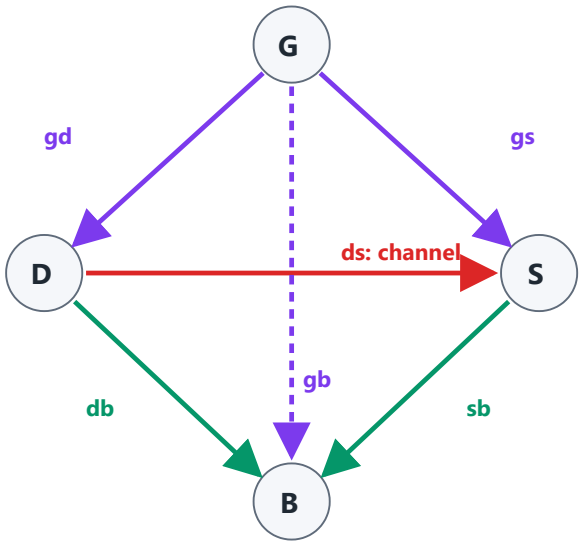


Figure 5.2. The six current paths between four terminals (the "abstract voltage-controlled non-linear bipole model" of slide 1). Red: channel. Purple: gate tunnelling. Green: junction / GIDL / impact ionisation.

ngspice can report BSIM4's *internal* current components (`@m1[igs]`, `@m1[igd]`, `@m1[ibd]`, ...). We record all eleven and combine them into six physical bipole currents. Kirchhoff then gives the terminals back exactly:

$$
\begin{aligned}
I_d &= I_{ds} - I_{gd} + I_{db} \\
I_g &= I_{gs} + I_{gd} + I_{gb} \\
I_s &= -I_{ds} - I_{gs} + I_{sb} \\
I_b &= -I_{gb} - I_{db} - I_{sb}
\end{aligned}
$$

Adding the four lines gives 0: Kirchhoff's law is satisfied *by construction* for every prediction, not just approximately.

PITFALLS WE HIT WHILE DECODING BSIM4

(1) When $V_d < V_s$, BSIM4 internally *swaps* drain and source, so `igcs` / `igcd` trade places and `isub` moves to the other side. (2) PMOS components are reported with n-type signs. Both were found by fitting terminal currents as signed sums of components (least squares) and confirmed by an automatic check: the six bipoles rebuild the four terminal currents to about 10^{-9} on every sample.

Every bipole is passive, so learn its conductance

For each bipole, current flows from the higher voltage to the lower one, like a resistor. We verified sign(I) = sign(V) on **100 % of all 40,000 samples**. So each bipole is a (very non-linear) resistor with a positive conductance:

$$I_{xy} = G_{xy}(\text{inputs}) \cdot V_{xy}, \quad G > 0 \;\Rightarrow\; \text{learn } \log G, \text{ then } I = e^{\log G} \cdot V$$

One refinement, explained in Chapter 6: the final model learns the conductance *per unit effective width*, $\log(G/W_{eff})$ with $W_{eff} = W - 2 \cdot \text{wint}$, and multiplies W_{eff} back in. Wider transistors carry proportionally more current, and building that in lets the network concentrate on everything else.

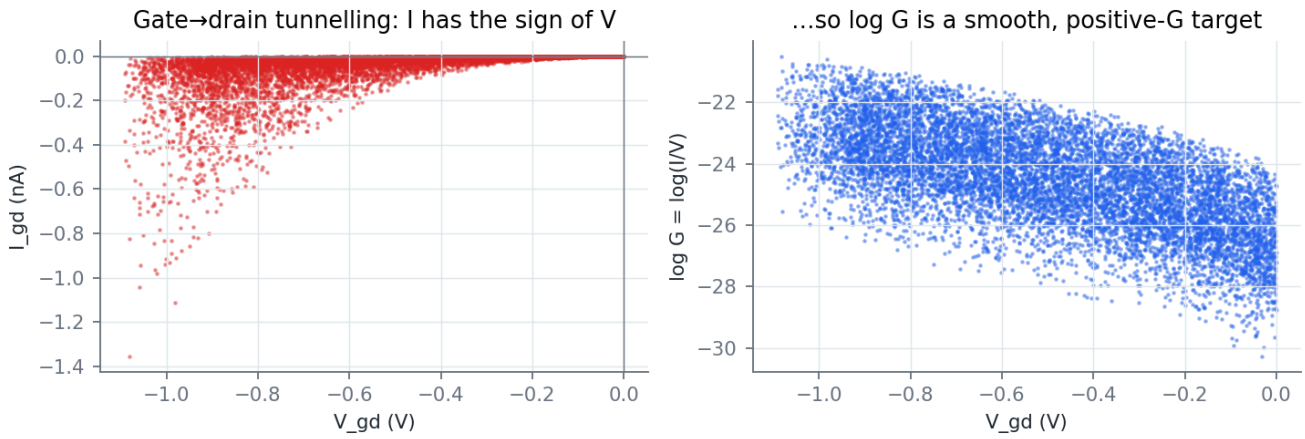


Figure 5.3. Left: gate-to-drain tunnelling current always has the sign of its voltage. Right: so log G is a smooth target with no zero crossings and no sign to predict. The sign and the zero come *for free* from multiplying by V.

Test symmetry, don't assume it

A MOSFET is physically symmetric: swap drain and source and nothing changes. If that held, gate→source and gate→drain tunnelling would be *the same function* with V_d, V_s swapped, and one model could learn both from twice the data. We tested it in SPICE by swapping V_d and V_s:

Bipole pair	Swaps exactly?	Design decision
gate↔source / gate↔drain tunnelling	✓ to every printed digit	One shared model, trained on both halves (2× data)
channel, gate↔body	✓ (symmetric by nature)	Features: lower and higher of (V_d, V_s)
drain↔body / source↔body junction	✗ drain ≈ 10^{-14} A, source ≈ 10^{-18} A whatever the voltages	Two separate models

Our prototype *had* assumed junction symmetry. Its junction model sat at 70-80 % error until we found this.

The final transistor model

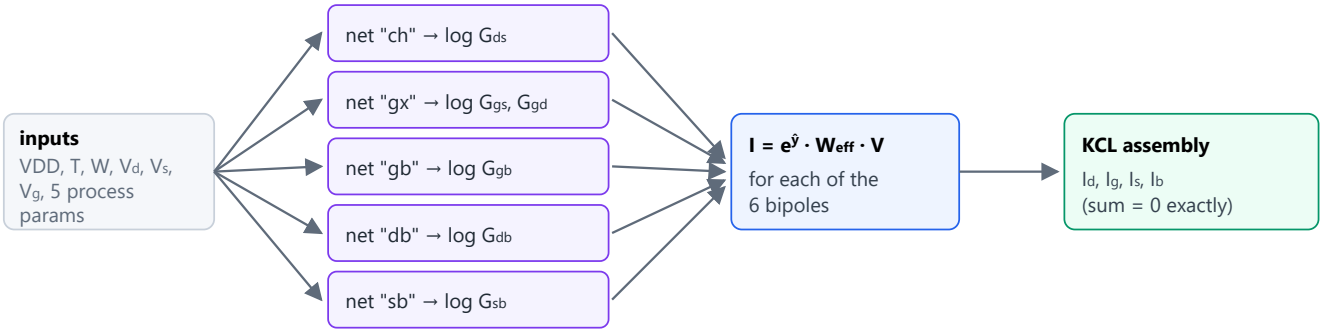


Figure 5.4. One TransistorModel (one per case, four in total). Five small regressors, one per mechanism, each predicting $\hat{y} = \log(G/W_{eff})$ from 13 features: VDD, T, $\log(W_{eff}/L)$, $(V_g$ − ideal level$)/V_T$, five normalised process parameters, the two diffusion voltages, V/V_T and $\log|V|$. Any of the four learner families can be plugged into the five slots, which is how the comparison in Chapter 6 is fair.

IN OUR PROJECT

`leakml/bipoles.py`: the six bipoles, the BSIM4 decoding, the KCL assembly. `leakml/models.py`: `TransistorModel`, the feature builder and the four learner families. Data: `data/<case>_bipoles.csv`.

VIVA CHECK

Your model predicts six things, but the brief asks for I_d, I_g, I_s, I_b. Is that allowed? Yes. The model's output *is* the four terminal currents, and every metric is computed on them. Internally it predicts the physical mechanisms and adds them with KCL. That is a modelling choice, like predicting log I instead of I, which the brief itself suggests.

What happens at V = 0, where G = I/V is 0/0? Below 1 mV we evaluate G at 1 mV and multiply by the true V. Any smooth I-V curve is linear near zero, so this is exact in the limit, and it gives I = 0 at V = 0, as physics requires.

6 Part A2: Training and choosing the models

Four families × two hyper-parameter settings × five mechanisms × four cases = 140 small models, trained in parallel in about 101 minutes.

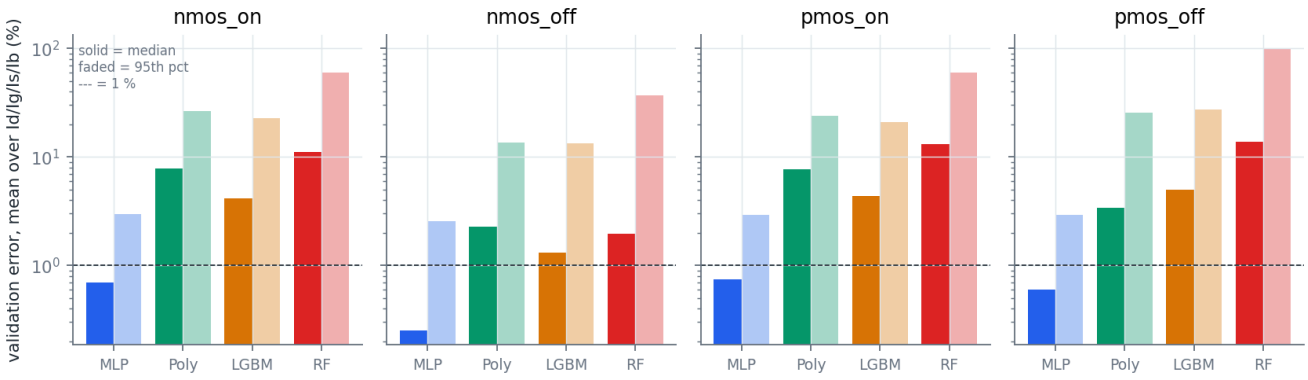


Figure 6.1. Validation error of the four families (best hyper-parameters per family, averaged over I_d, I_g, I_s, I_b; log scale). The neural network is best in every case, 4-7× better than the next family (lower score). This is the staircase-vs-smooth story of Figure 4.2, now on the real 13-dimensional problem.

Model comparison table (deliverable 4, validation set)

case	family	best h-params	R^2 Id	R^2 Ig	Id med / p95	Ig med / p95	Is med / p95	Ib med / p95
nmos_on	**Neural net (MLP)**	**(64, 64, 64)**	**1.0000**	**1.0000**	**0.91 / 4.1**	**0.38 / 1.5**	**0.91 / 4.1**	**0.61 / 2.1**
nmos_on	LightGBM	num_leaves=31	1.0000	0.9994	2.66 / 26.0	3.93 / 12.6	2.68 / 25.9	7.24 / 25.8
nmos_on	Polynomial + Ridge	degree=4	1.0000	0.9960	6.59 / 23.8	9.53 / 32.1	6.59 / 23.8	8.65 / 25.8
nmos_on	Random Forest	trees=300	0.9998	0.9958	7.11 / 62.3	9.47 / 33.5	7.11 / 61.8	20.34 / 82.0
nmos_off	**Neural net (MLP)**	**(64, 64, 64)**	**1.0000**	**1.0000**	**0.28 / 3.8**	**0.15 / 0.5**	**0.27 / 4.0**	**0.29 / 1.9**
nmos_off	LightGBM	num_leaves=31	0.9981	1.0000	1.59 / 21.4	0.77 / 2.2	1.49 / 21.6	1.38 / 8.3
nmos_off	Polynomial + Ridge	degree=4	0.9999	1.0000	0.05 / 3.9	0.01 / 0.0	0.08 / 4.0	8.99 / 46.5
nmos_off	Random Forest	trees=300	0.9869	0.9999	2.49 / 63.0	0.93 / 3.7	2.66 / 62.0	1.69 / 18.1
pmos_on	**Neural net (MLP)**	**(64, 64, 64)**	**1.0000**	**1.0000**	**0.87 / 3.7**	**0.64 / 2.2**	**0.87 / 3.7**	**0.58 / 2.0**
pmos_on	LightGBM	num_leaves=31	1.0000	0.9990	2.17 / 18.6	6.86 / 24.2	2.17 / 18.5	6.19 / 22.6
pmos_on	Polynomial + Ridge	degree=4	1.0000	0.9976	6.48 / 20.6	8.75 / 27.1	6.48 / 20.6	8.98 / 27.6
pmos_on	Random Forest	trees=300	0.9999	0.9913	6.08 / 43.7	21.37 / 80.2	6.08 / 43.7	18.53 / 70.4
pmos_off	**Neural net (MLP)**	**(64, 64, 64)**	**1.0000**	**0.9936**	**0.64 / 3.7**	**0.33 / 1.7**	**0.96 / 4.1**	**0.47 / 2.3**
pmos_off	LightGBM	num_leaves=31	0.9992	0.9935	6.50 / 44.9	1.19 / 3.3	10.66 / 49.3	1.70 / 11.7
pmos_off	Polynomial + Ridge	degree=4	0.9999	0.9936	1.52 / 14.1	0.22 / 1.6	0.99 / 23.3	10.75 / 63.5
pmos_off	Random Forest	trees=300	0.9918	0.9934	21.54 / 164.4	1.53 / 6.0	29.60 / 199.7	2.71 / 23.7

Median and p95 are relative errors in %, for currents above 10^{-14} A. R^2 is on a signed-log scale. Highlighted: the chosen model per case.

Why the ranking comes out this way

- **MLP wins** because the targets (log G) are smooth functions of 13 inputs, and a tanh network is a smooth universal approximator. L-BFGS trains it to high precision on noise-free data.
- **Polynomial + Ridge** is smooth too, but a degree-4 polynomial in 13 variables has 2,380 terms and still cannot bend sharply enough near $V \approx 0$ or across temperature.
- **LightGBM and Random Forest** are excellent on tabular data with thresholds and categories, but here they approximate a smooth surface with flat boxes. That gives large p95 errors, especially in the steep regions.

Chosen models on the untouched test set (deliverable 5)

case	chosen model	Id med / p95 (kept)	Ig	Is	Ib	rail test: Id med / p95
nmos_on	Neural net (MLP) (64, 64, 64)	0.86 / 4.2 (100 %)	0.39 / 1.6 (100 %)	0.86 / 4.1 (100 %)	0.61 / 2.1 (100 %)	0.92 / 4.6
nmos_off	Neural net (MLP) (64, 64, 64)	0.29 / 3.8 (100 %)	0.16 / 0.5 (100 %)	0.28 / 3.5 (100 %)	0.33 / 1.8 (88 %)	0.64 / 6.5
pmos_on	Neural net (MLP) (64, 64, 64)	0.84 / 3.5 (100 %)	0.64 / 2.4 (94 %)	0.84 / 3.5 (100 %)	0.59 / 2.1 (100 %)	0.90 / 4.7
pmos_off	Neural net (MLP) (64, 64, 64)	0.58 / 3.5 (94 %)	0.38 / 1.6 (31 %)	0.93 / 4.0 (76 %)	0.48 / 2.4 (72 %)	1.00 / 6.4

"kept" is the share of test samples whose current exceeds 10^{-14} A (the rest are numerical noise, Chapter 3). PMOS-OFF gate current is almost entirely below the floor: an OFF PMOS has $V_g = V_b = VDD$, so very little voltage sits across its oxide.

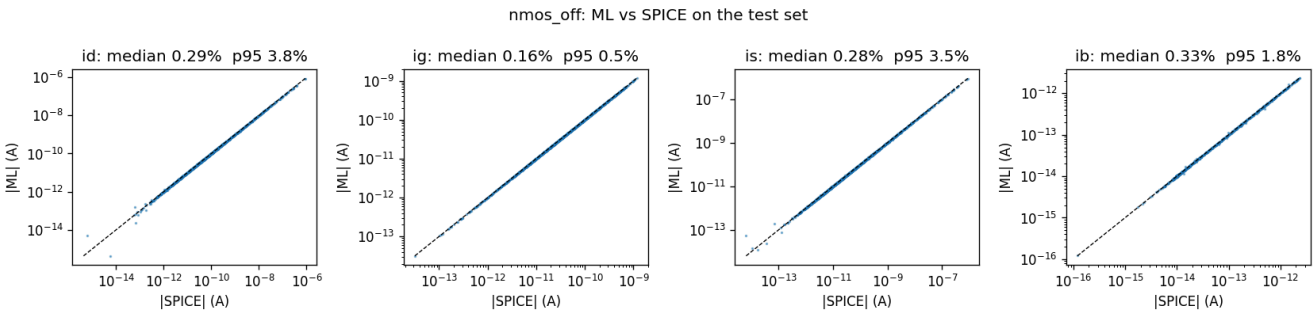


Figure 6.2. Predicted vs SPICE, NMOS OFF, test set (the other three cases are in `figures/pred_vs_spice_*.png`). Points on the dashed line are perfect. The spread at the bottom-left is the numerical-noise region.

Three versions of the model: a debugging story

The model above is the *third* version. The first two had excellent transistor test scores and still failed at the gate level. How we found and fixed each problem is the most useful part of this project to understand.

Version	What changed	gate median	gate p95	adder median
v1	10k uniform samples per case, features: W/L	4.26 %	17.81 %	–
v2	+ 5k rail-focused samples per case	2.01 %	8.36 %	4.97 %
v3	+ effective-width target and feature, + V_g offset feature (final)	1.40 %	5.55 %	2.85 %

Version 1 → 2: distribution shift

Version 1 was trained on the 10,000 uniform samples only. Transistor test error ≈ 0.7 % median, yet gate median **4.26 %**, p95 **17.81 %**, and four gate cases biased by +10 %, all of them dominated by one OFF NMOS with $V_s = 0$ and $V_d \approx VDD$.

> **INTUITION: DISTRIBUTION SHIFT**
>
> Imagine learning to drive only on quiet suburban streets, then taking the test on a motorway. You were trained well, just not *where you are tested*. In a real gate the transistor that leaks has its source **exactly at 0 V** (or VDD) and its drain at the output, **almost exactly at VDD** (or 0). Uniform sampling of V_d, $V_s \in [0, VDD]$ almost never lands there: only **3 of 10,000** samples were within 2 % of both rails. There the model was extrapolating, with 3.7 % median error against 0.45 % in the interior.

Fix: the brief allows "the number of samples may be increased as needed". We kept the required 10,000 uniform samples as the deliverable and added a separate **rail-focused supplement of 5,000 samples per case**: each of V_d, V_s is exactly 0 or VDD one third of the time, and otherwise follows an *arcsine* distribution (Beta(½,½)), which crowds samples near both rails but still covers the middle. Model selection averages the uniform and rail validation scores; test numbers are still reported on the untouched uniform test set.

Version 2 → 3: a "transistor oracle" finds two hidden causes

Version 2 halved the gate error, but two systematic patterns remained: some gates +3 % too high, AND2 "11" and OR2 "00" 6 % too low. To find out *which transistor* was responsible, we built a diagnostic: take every transistor inside the gate, re-simulate it **alone** in SPICE at the exact terminal voltages it has inside the gate, and compare its power with the ML prediction, device by device.

Gate	Culprit transistor	Its condition	ML error	Cause
AND2 11	inverter NMOS (OFF)	V_g = 16 mV, not 0	−27 %	model had no V_g input
INV 0	NMOS (OFF)	V_s = 0, V_d ≈ VDD, W = 2L	+3.6 %	width effect poorly learnt for narrow devices

INTUITION: TWO PHYSICS FEATURES THE NETWORK WAS MISSING

Gate voltage. In AND2 the inverter is driven by the NAND output, a "logic 0" that actually sits at 16-60 mV. Sub-threshold current is exponential in V_{GS}: 16 mV/(1.5·26 mV) ≈ 0.4 → $e^{0.4}$ ≈ 1.5× more current. Our cases fixed V_g at exactly 0 or VDD, so the model could not see this. Fix: add $(V_g - \text{ideal})/V_T$ as a feature, and in the supplement move V_g up to 10 % of VDD away from its rail half the time.

Effective width. BSIM4 trims `wint` = 5 nm from each edge of the channel, so a drawn W = 16 nm conducts like 6 nm and W = 32 nm like 22 nm. Current is proportional to W_{eff}, not W. At the narrow end the relationship is strongly curved, and the network, given only W/L, under-fitted it in a wavy pattern: +5.3 % at W = 1.5L, +2.8 % at 2L, −2.6 % at 3L. Fix: predict conductance *per unit effective width* and feed $\log(W_{eff}/L)$. The bias dropped to within ±1.5 % at every width.

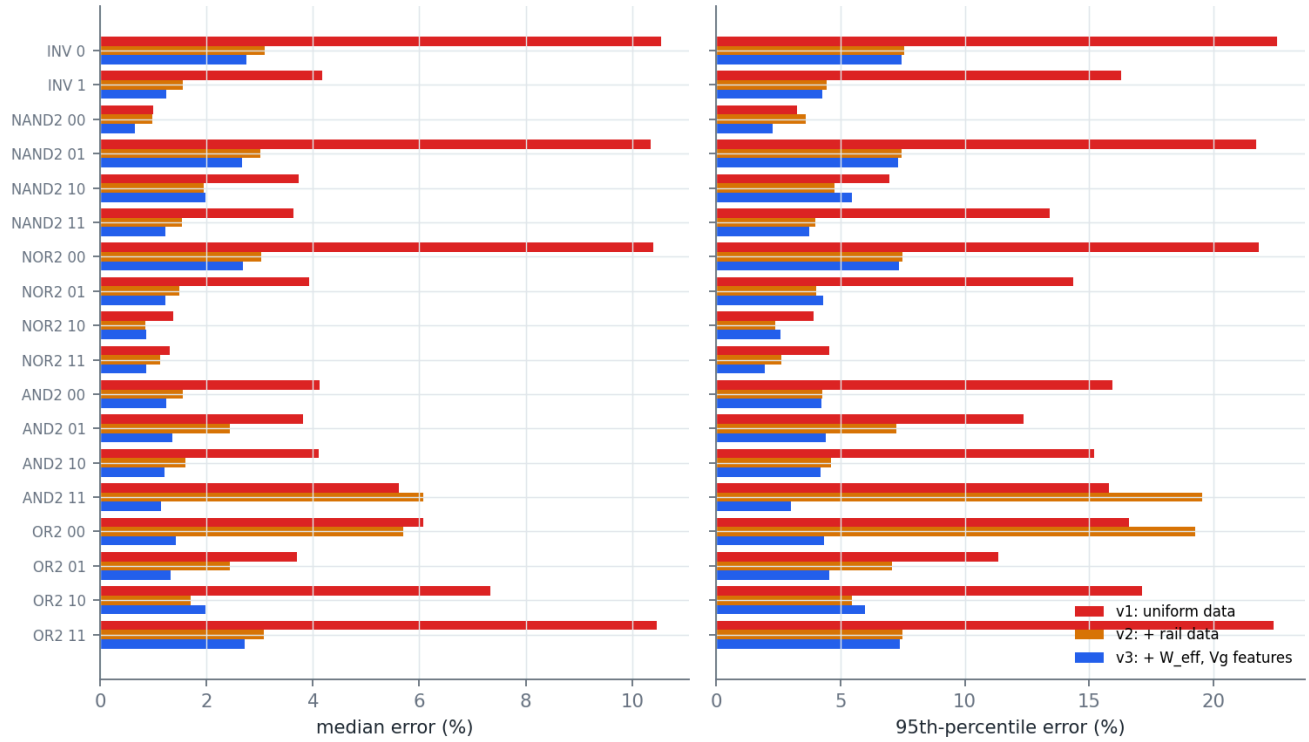


Figure 6.3. Gate-level error of the three model versions, for all 18 gate cases. Same gates, same 200 corners; only the transistor models changed.

> **LESSON**
>
> A great test-set score is necessary but not sufficient. The test set measures accuracy *where the data was drawn*; the application decides where accuracy is *needed*. Every fix here came from an error pattern, a hypothesis grounded in device physics, and a targeted check, not from trying bigger models.

Ablation: how much does the bipole idea buy?

Same neural network and the same uniform training data, but predicting the four terminal currents *directly* (on an asinh signed-log scale), against our bipole model (validation set, median / p95 %). Note the bipole model here also had the rail data; the point is the size of the gap.

case	current	direct MLP	bipole MLP (ours)
nmos_on	id	6.42 / 29.5	0.91 / 4.1
nmos_on	ig	1.08 / 5.5	0.38 / 1.5
nmos_on	is	6.90 / 30.4	0.91 / 4.1
nmos_on	ib	1.05 / 4.3	0.61 / 2.1
nmos_off	id	2.91 / 18.8	0.28 / 3.8
nmos_off	ig	0.31 / 1.3	0.15 / 0.5
nmos_off	is	3.06 / 19.3	0.27 / 4.0
nmos_off	ib	0.30 / 6.7	0.29 / 1.9
pmos_on	id	5.98 / 25.5	0.87 / 3.7
pmos_on	ig	1.17 / 4.4	0.64 / 2.2
pmos_on	is	5.90 / 28.0	0.87 / 3.7
pmos_on	ib	1.57 / 6.3	0.58 / 2.0
pmos_off	id	2.81 / 13.8	0.64 / 3.7
pmos_off	ig	1.61 / 7.5	0.33 / 1.7
pmos_off	is	3.82 / 20.6	0.96 / 4.1
pmos_off	ib	0.60 / 10.3	0.47 / 2.3

Learning curve: how much data do we need?

The brief asks for high accuracy 'with the smallest dataset possible'. We retrained the chosen models on growing subsets of the training data. Average median error: 500 samples → 3.85 %, 1,000 samples → 1.32 %, 2,000 samples → 0.80 %, 4,000 samples → 0.62 %, 7,000 samples → 0.57 %. (Each size n also gets n/2 rail samples, the same mix as the full model; 7,000 is the whole training split.) The error falls steeply up to about 2,000 samples and then flattens: the last 3,000 samples buy only a few hundredths of a percent. So 10,000 samples per case is comfortably enough, and about 4,000 would already give sub-1 % median error.

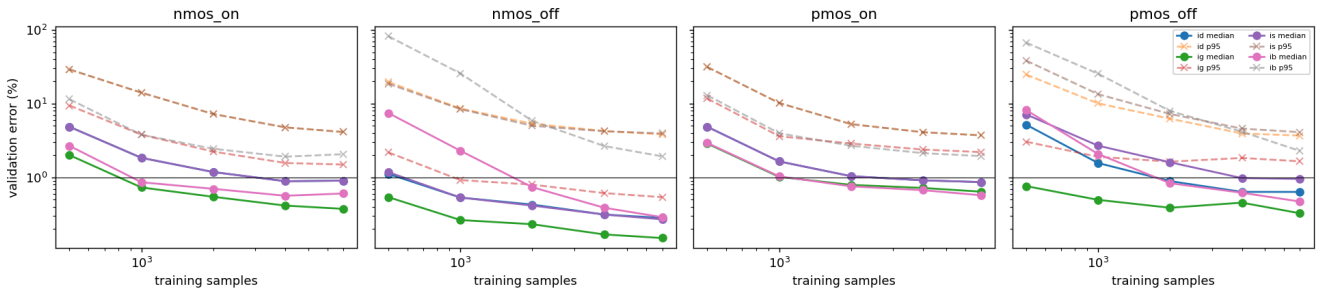


Figure 6.4. Learning curve. Validation error vs number of training samples, per case and current. Where the curves flatten, more data stops helping.

VIVA CHECK

Did adding rail samples "cheat" by using test information? No. The rail samples are new random single-transistor simulations; no gate or circuit result was used. We used *knowledge of where the model will be applied* (terminals at the rails), which is standard practice: match the training distribution to the deployment distribution.

Why is R^2 almost 1 for every model when their errors differ by 10×? R^2 measures the share of variance explained. Our targets vary over many decades, so even a 10 %-error model explains more than 99 % of the variance. That is why median and p95 relative errors are the metrics that discriminate between models.

7 Part B: From transistors to gates

5 gates × all input combinations = 18 cases, each at 200 random PVT corners. SPICE simulates each full gate; the ML adds up its transistors.

The gates and their internal nodes

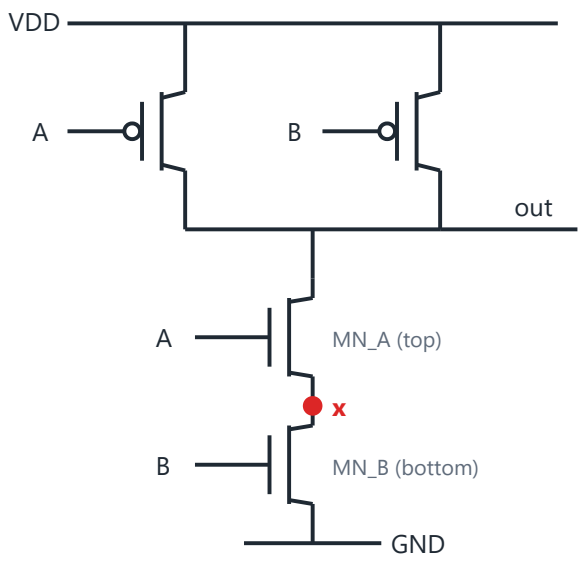


Figure 7.1. NAND2. Internal node x sits between the two series NMOS. NOR2 is the mirror image (node y between series PMOS). AND2 = NAND2 + INV and OR2 = NOR2 + INV, simulated as complete gates.

Sizing: $W_n = 2 \cdot L_{min} = 32$ nm, $W_p = 4 \cdot L_{min} = 64$ nm. Both lie *inside* the 1-8·L_{min} training range (not at its edge, where models are weakest), and PMOS is twice NMOS as is usual for static CMOS.

A corner = one random draw of VDD, T, one set of NMOS process parameters and one set of PMOS parameters (all NMOS in a gate share one draw, all PMOS another). The *same 200 corners* are reused for every gate and for the adder.

How the ML predicts a gate: for each transistor, take its four terminal voltages from the SPICE node voltages (rails, inputs, x/y, output), choose the model from its gate input's logic value (ON/OFF), predict the six bipole currents, and add up.

What exactly is "gate leakage"? A decision we had to make

The obvious answer is "the current drawn from VDD". But gate tunnelling also flows *into the input pins*. In the isolated-gate test an ideal source supplies it; in a real circuit the previous gate does, from its own VDD. If we count only I(VDD), gate leakages don't add up to circuit leakage. So we define:

$$I_{leak} = P_{static} / VDD, \quad P_{static} = \text{total DC power delivered by all sources (VDD rail and every input driver)}$$

This is the standard "equivalent leakage current" ($P_{leak} = VDD \cdot I_{leak}$). Defined this way, gate leakages add up to the circuit's. We also report plain I(VDD) for completeness.

GOING DEEPER: TELLEGEN'S THEOREM MAKES THE ML SIDE ROBUST

Power delivered by the sources equals power absorbed by the devices (Tellegen's theorem). For a transistor, the absorbed power is the sum over its six bipoles of $V \cdot I = G \cdot V^2 \geq 0$. So:

$$P_{gate} = \Sigma_{transistors} \Sigma_{bipoles} G_{xy} \cdot V_{xy}{}^2$$

Why this is clever: an ON transistor in a leaking stack carries the leakage current across a *tiny* V_{ds} (a few mV). Its channel current is hard to predict accurately, but its power is that current × a few mV, so any error is multiplied by almost zero. The formula automatically weights each prediction by how much it matters. We never need any transistor's I_s directly.

The stack effect: real physics in our data

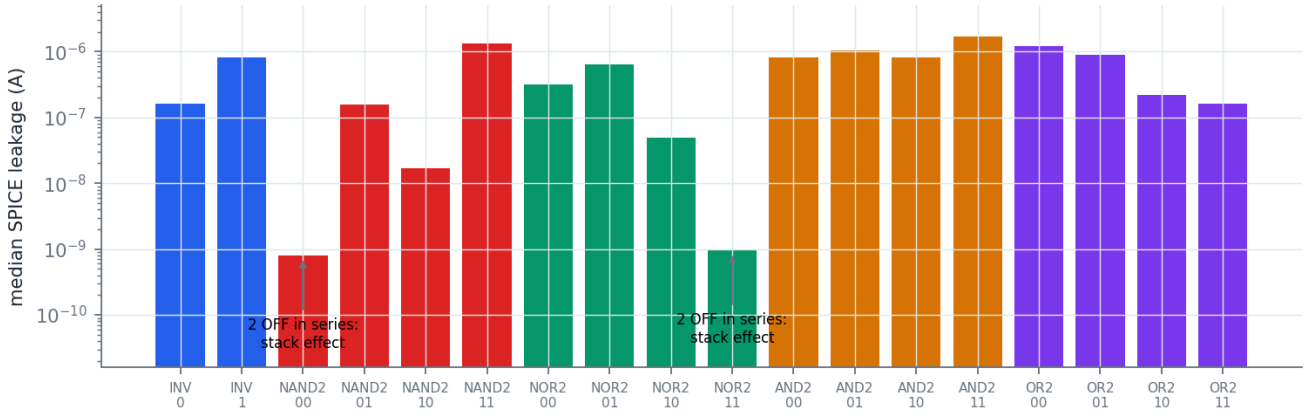


Figure 7.2. Median SPICE leakage of all 18 gate cases. Leakage spans about three decades depending on the *input pattern*. The two lowest bars are NAND2 "00" and NOR2 "11": two OFF transistors in series.

INTUITION: WHY TWO OFF TRANSISTORS LEAK FAR LESS THAN ONE

In NAND2 with AB = 00, both NMOS are OFF. The tiny leakage through the top one charges node x up to about 0.17 V (median of our 200 SPICE corners) until the two leakages balance. Now the top transistor's source is at 0.18 V instead of 0, which hurts it three ways: $V_{GS} = -0.18$ V (exponentially less current), body effect ($V_{SB} > 0$ raises V_{th}), and less V_{DS} (less DIBL). Result: NAND2 "00" leaks about 200× less than NAND2 "01", where a single OFF NMOS blocks the path. This is why "input vector control" (parking idle circuits in their lowest-leakage input state) is a real power-saving technique.

Gate results table (deliverable 6)

gate	inputs	SPICE median (A)	ML median (A)	bias %	median err %	p95 err %	I(VDD) med err %
INV	0	1.61×10^{-7}	1.67×10^{-7}	+2.32	2.75	7.44	3.12
INV	1	8.08×10^{-7}	8.08×10^{-7}	-0.24	1.24	4.28	1.26
NAND2	00	7.92×10^{-10}	7.88×10^{-10}	-0.05	0.66	2.29	1.69
NAND2	01	1.57×10^{-7}	1.62×10^{-7}	+2.23	2.67	7.32	3.24
NAND2	10	1.72×10^{-8}	1.77×10^{-8}	+1.98	1.98	5.47	4.02
NAND2	11	1.34×10^{-6}	1.34×10^{-6}	-0.48	1.23	3.74	1.32
NOR2	00	3.18×10^{-7}	3.27×10^{-7}	+2.26	2.68	7.36	2.57
NOR2	01	6.48×10^{-7}	6.54×10^{-7}	-0.20	1.23	4.30	1.70
NOR2	10	4.92×10^{-8}	4.96×10^{-8}	+0.43	0.86	2.60	1.08
NOR2	11	9.47×10^{-10}	9.58×10^{-10}	+0.37	0.87	1.97	1.01
AND2	00	8.08×10^{-7}	8.09×10^{-7}	-0.29	1.24	4.24	1.27
AND2	01	1.04×10^{-6}	1.05×10^{-6}	+0.15	1.36	4.40	1.25
AND2	10	8.25×10^{-7}	8.26×10^{-7}	-0.21	1.22	4.19	1.21
AND2	11	1.70×10^{-6}	1.69×10^{-6}	-0.02	1.15	3.01	1.05
OR2	00	1.22×10^{-6}	1.22×10^{-6}	+0.29	1.43	4.34	1.38
OR2	01	8.83×10^{-7}	8.83×10^{-7}	+0.52	1.32	4.54	1.82
OR2	10	2.18×10^{-7}	2.21×10^{-7}	+1.84	1.98	6.00	2.42
OR2	11	1.62×10^{-7}	1.68×10^{-7}	+2.31	2.72	7.39	3.10

SPICE / ML median in A; bias = median signed error; errors are |ML−SPICE|/SPICE over 200 corners. Last column: median error if leakage is defined as plain I(VDD).

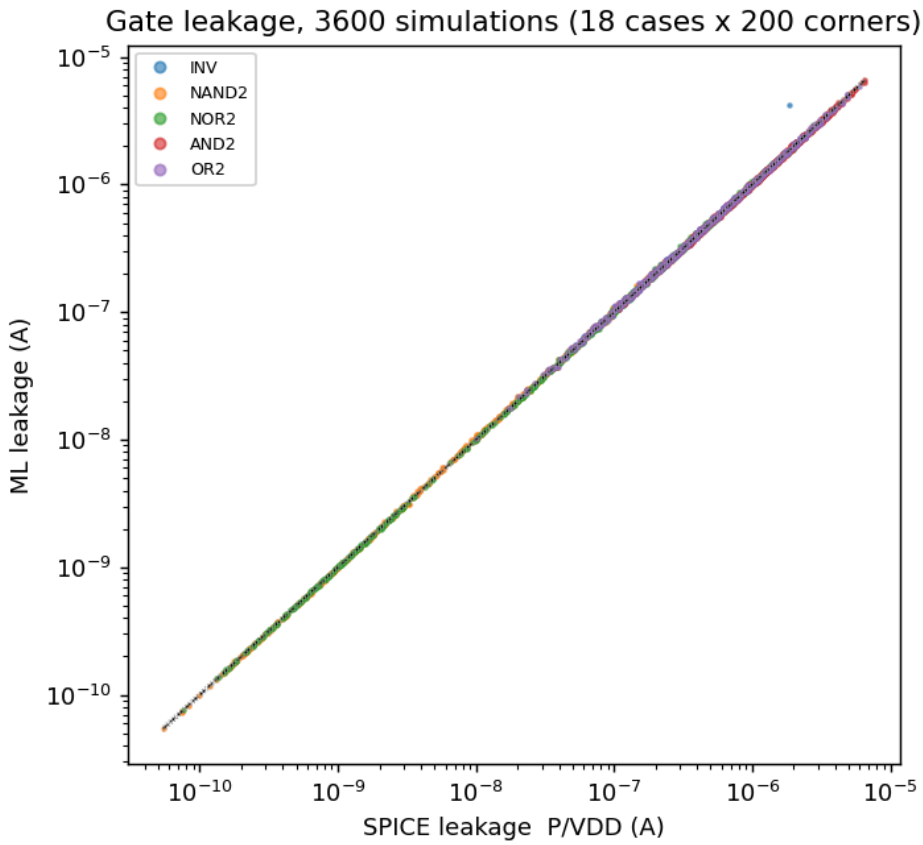


Figure 7.3. Gate parity plot (deliverable 7): all 3,600 gate simulations on log axes. Dotted lines mark ±1 % and ±5 %.

Most and least accurate: most accurate (median / p95): NOR2 11 (0.87 % / 1.97 %), NAND2 00 (0.66 % / 2.29 %), NOR2 10 (0.86 % / 2.60 %). Least accurate: INV 0 (2.75 % / 7.44 %), OR2 11 (2.72 % / 7.39 %), NOR2 00 (2.68 % / 7.36 %). Over all 3,600 simulations: median 1.40 %, p95 5.55 %, mean accuracy 98.02 %.

IN OUR PROJECT

`04_gate_validation.py` · topology shared by SPICE and ML in `leakml/circuits.py` (so they can never disagree) · leakage definition and the Σ $G \cdot V^2$ evaluation in `leakml/evaluate.py` · outputs `results/gate_results.csv`, `results/gate_samples.csv` (every simulation with SPICE node voltages), `netlists/gate_*.cir`.

8 Part C: The 3-bit carry-lookahead adder

42 gates, 204 transistors, 7 inputs → 128 input vectors × 200 corners = 25,600 full-circuit SPICE simulations, against a sum of 42 gate predictions.

What a carry-lookahead adder is

A ripple adder waits for each carry to pass from bit to bit. A **carry-lookahead** adder computes every carry *directly* from the inputs using two signals per bit: **generate** $g_i = a_i \cdot b_i$ (this bit makes a carry by itself) and **propagate** $p_i = a_i \oplus b_i$ (this bit passes an incoming carry on).

$$c_1 = g_0 + p_0 c_0$$
$$c_2 = g_1 + p_1 g_0 + p_1 p_0 c_0$$
$$c_3 = g_2 + p_2 g_1 + p_2 p_1 g_0 + p_2 p_1 p_0 c_0 \quad \text{(carry out)} \qquad s_i = p_i \oplus c_i$$

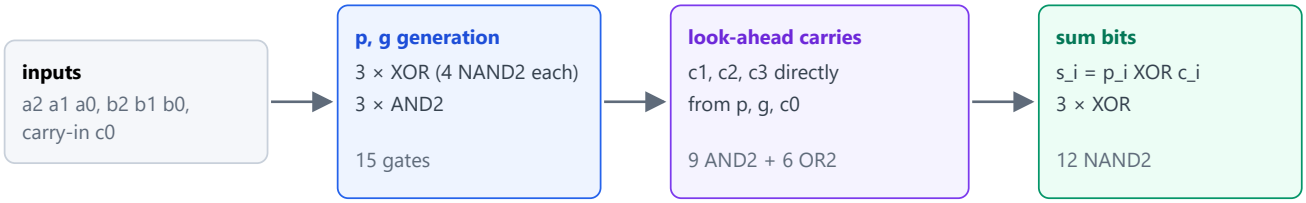


Figure 8.1. Our 3-bit CLA, built only from the five library gates: 24 NAND2 + 12 AND2 + 6 OR2 = 42 gates, 204 transistors. XOR uses the classic four-NAND2 structure. A script checks that it adds correctly for all 128 input vectors.

Hierarchical estimation: the method of the brief

1. For an input vector, propagate the logic values through the 42 gates (pure Boolean logic, no SPICE).
2. For each gate, look up its ML leakage from Part B for *its* input combination at this corner.
3. Add the 42 numbers. That is the circuit's leakage.

This assumes every gate input sits *exactly* at 0 or VDD and that gates don't load each other. Neither is quite true: from our SPICE data, a "0" output actually sits at a few mV (inverter, median 6 mV) to about 20 mV (NAND2, median 21 mV, up to 100 mV at the hottest corners), because the OFF PMOS leaks through the ON NMOS.

Splitting the error into its causes

A single error number can't tell you *why* the answer is off. So we computed three comparisons that separate the two sources of error:

Comparison	What it isolates	median	p95	bias
Hierarchical ML vs SPICE circuit	the full method of the brief	2.85 %	11.32 %	-2.69 %
SPICE gate sum vs SPICE circuit	error of the gate-level assumption itself	2.64 %	10.04 %	-2.64 %
ML gate sum vs SPICE gate sum	error of the ML models alone	1.26 %	3.70 %	+0.01 %
Flat ML (circuit node voltages) vs SPICE circuit	transistor models without any gate-level assumption	1.21 %	3.54 %	+0.01 %

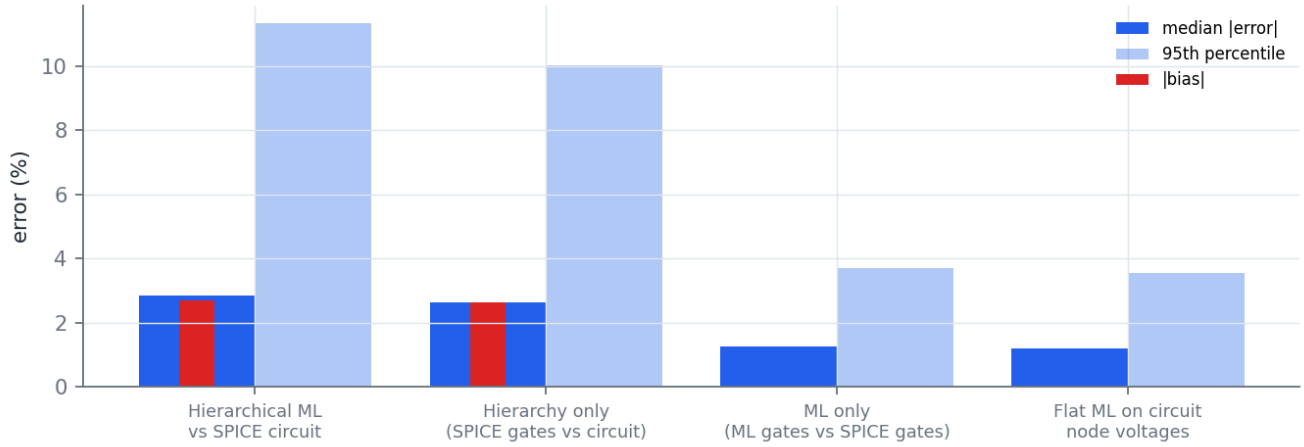


Figure 8.2. Where the adder error comes from. "Hierarchy only" adds up *SPICE* gate leakages, so the ML plays no part: whatever error remains is the price of treating the circuit as independent ideal gates. "ML only" compares ML gates with SPICE gates. "Flat ML" runs the transistor models directly on the adder's own SPICE node voltages, with no gate-level assumption.

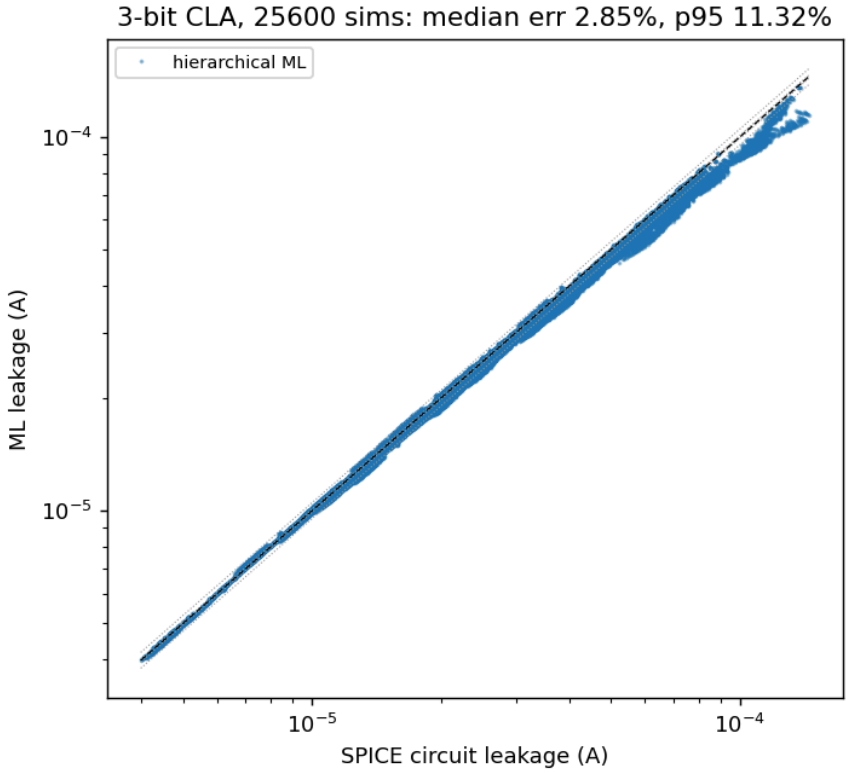


Figure 8.3. Test-circuit parity plot (deliverable 8): hierarchical ML vs full-circuit SPICE for all 25,600 simulations, log axes.

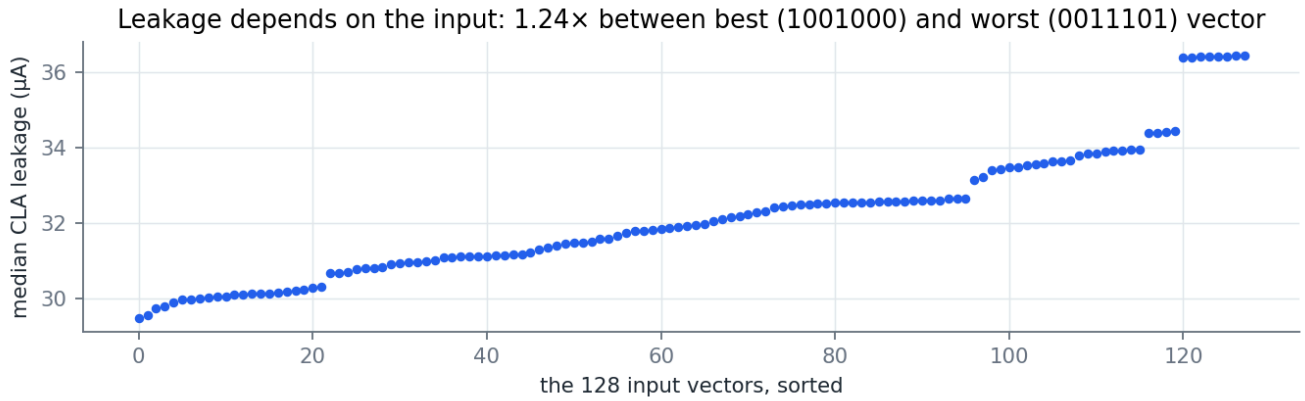


Figure 8.4. Bonus insight: the adder's leakage depends on its input. Median over the 200 corners for each of the 128 vectors: the stack effect at circuit scale.

IN OUR PROJECT

`05_circuit_validation.py` · CLA definition and logic check in `leakml/circuits.py` (`cla3_gates`) · outputs `results/circuit_results.csv` (per input vector), `results/circuit_summary.csv`, `netlists/cla3.cir`.

9 Error analysis, limitations, improvements

Where the largest errors occur, and why

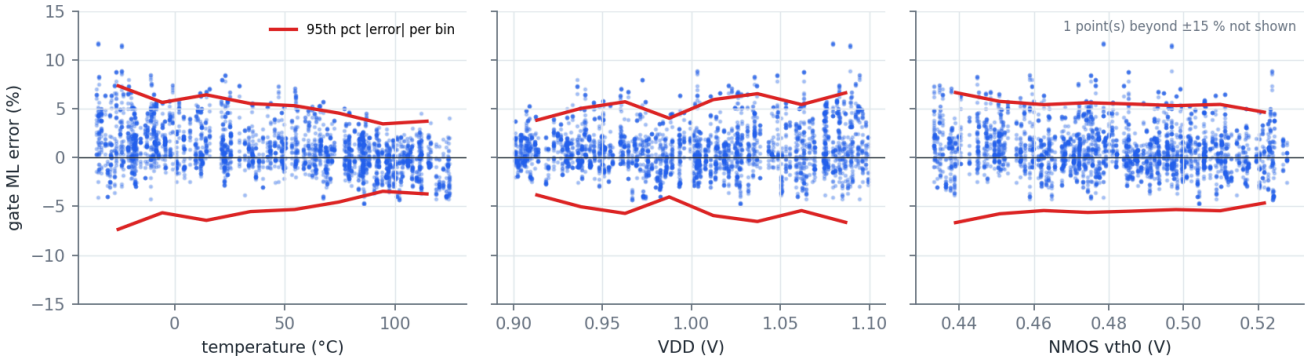


Figure 9.1. Gate-level ML error against the corner conditions. Red lines: 95th percentile of |error| in each bin. Use this to answer "is the error worse at hot / cold / low-vth0 corners?"

subset of gate simulations	median |err| %	p95 |err| %
all gate sims	1.40	5.55
T > 85 °C	1.23	3.63
T < 0 °C	2.00	6.71
lowest 20 % NMOS vth0	1.73	6.39
VDD > 1.07 V	2.08	6.27
VDD < 0.93 V	1.13	3.74

By gate and input: the largest p95 errors are INV 0 (7.4 %), OR2 11 (7.4 %), NOR2 00 (7.4 %), NAND2 01 (7.3 %).

Reading the numbers

- **Cold and high-VDD corners are the hardest; hot corners are the easiest.** Below 0 °C the median error is 2.0 % (bias +1.7 %), above 85 °C it is 1.2 %. When it is cold the sub-threshold slope is steepest, so log I is most sensitive to every input and a small error in any feature is amplified. At high VDD (above 1.07 V) DIBL is strongest, and VDD = 1.1 V is the edge of the training range. The worst gate samples (about 12 %) all sit at two corners that combine both: −35 °C at 1.08 V and −24 °C at 1.09 V.
- **A small leftover bias of about +2 %** appears in INV 0, NAND2 01, NOR2 00 and OR2 11. Every one of them is dominated by the same device: an OFF NMOS with $V_s = 0$ and $V_d \approx$ VDD at W = 2L. This is the condition the effective-width fix (Chapter 6) improved from +3.6 %. The rest of that bias is close to the model's own transistor-level error (below 1 % median), but because every corner sees it in the same direction, it shows up as a bias and not as noise.
- **One isolated outlier (+126 %, 1 of 3,600).** INV with input 1 at corner 198 (112 °C, 1.09 V). Here SPICE itself looks inconsistent: inside AND2, at the same corner, the same inverter state leaks 3.58 µA with its output at 42 mV, while the standalone inverter gives only 1.85 µA at 22 mV. The ML predicts 4.18 µA, closer to the AND2 value. We report it unchanged. Without it the maximum gate error is about 12 %.

Why the adder is less accurate than the gates

The decomposition in Chapter 8 answers this directly. Adding up *SPICE* gate leakages already misses the real adder by −2.6 % (median), with no ML involved. The ML part adds only about 1.3 % median error on top. The bias is negative because real logic levels are not ideal: a "0" sits a few to a hundred millivolts above ground and a "1" a little below VDD. Each such input turns the next gate's OFF transistor slightly *on* (exponentially, through V_{GS}), so the real circuit leaks more than a table of ideal-input gates predicts.

> **THE PROOF: FLAT ML**
>
> When the same transistor models are run on the adder's real node voltages ("flat ML"), the error drops to 1.2 % median with essentially zero bias. In version 2 this number was 5.2 %. The difference is the V_g-offset feature from Chapter 6: the v3 models *can* see a non-ideal input level and price it correctly. So the remaining adder error belongs to the ideal-level hierarchy, not to the ML, and a "levels-aware" hierarchy (next section) is the obvious next step.

Limitations

- **Ideal-level assumption (Part C).** The hierarchy assumes inputs at exactly 0/VDD. At 16 nm, 1 V, a logic 0 can sit at 60-100 mV at hot corners, enough to change the next gate's leakage by several percent. This is a property of the *method*, not the ML.
- **Gate voltage only near its logic level.** The models take V_g as an input, but were trained only within 10 % of VDD of the ideal level. A gate driven by a badly degraded signal would be outside the training range.
- **Tiny currents.** Below $\approx 10^{-14}$ A the SPICE data is numerical noise, so no model can be judged there (it doesn't matter for leakage totals).
- **One model card, one L.** Models are valid for 16 nm HP, L = L_{min}, W in 1-8·L_{min}, inside the trained PVT ranges. Outside them they extrapolate.
- **Junction model artefact.** No AS/AD/PS/PD are given, so junction leakage is a geometry-default constant. Real layouts would leak more through junctions.

What we would do next

- **ML node voltages.** Solve each gate's internal node voltages with the ML models themselves (KCL root-finding: every bipole is monotonic, so bisection always converges). That removes SPICE from the prediction completely.
- **Non-ideal inputs in Part C.** Feed each gate its driver's predicted output level instead of exact 0/VDD (a "levels-aware" hierarchy). That attacks the dominant Part C error directly.
- **Ensembles.** Average several MLPs trained from different random starts, which cuts variance in the tails at little cost.
- **Active learning.** Add SPICE samples where the model is least certain, for the same accuracy with fewer simulations.

10 Viva preparation

Short, confident answers. Each one points to evidence you can show.

Q. Walk me through your project in one minute.

> We simulate a single 16 nm transistor 40,000 times across random process, voltage and temperature and train four ML models (NMOS/PMOS × ON/OFF) to predict its terminal currents. A gate's leakage is then the sum over its transistors, and the adder's is the sum over its 42 gates. We check both against full SPICE: median error 1.40 % for gates and 2.85 % for the adder.

Q. Why log scale?

> Currents span 14 decades. On a linear scale the squared error is dominated by the largest currents. On a log scale a fixed error means a fixed percentage for every current, which is what we are graded on.

Q. Why did you not predict Id, Ig, Is, Ib directly?

> Each terminal current is a sum of mechanisms with opposite signs that cancel and create sharp zero-crossings (Figure 5.1). We predict each mechanism (bipole), which is smooth and one-signed, as log G, and recombine with Kirchhoff's law. The ablation table shows the gain.

Q. How do you know the bipoles are right?

> Two automatic checks: they rebuild the four SPICE terminal currents to about 1e-9 on every sample, and sign(I) = sign(V) holds for 100 % of samples.

Q. Why a neural network and not XGBoost/LightGBM?

> Tree models predict piecewise constants, which is good for thresholds but poor for smooth exponential laws (Figure 4.2). On our validation set the MLP's error score was 4-7× lower than the next-best family in every case.

Q. What is L-BFGS?

> A quasi-Newton optimiser: it uses the whole dataset each step and estimates curvature to take well-aimed steps. It suits small, noise-free problems like ours better than Adam.

Q. How did you choose the model? Did you look at the test set?

> Every model was compared on the validation set only. The test set was used once, at the end, for the chosen model. The split is 70/15/15 with seed 42.

Q. Why does a 10 % vth0 change give 3× leakage?

> Sub-threshold current is ∝ exp(−Vth/(n·kT/q)). 48 mV/(1.5 × 26 mV) ≈ 1.2 and e^(−1.2) ≈ 0.3.

Q. What is the stack effect? Show it.

> Two series OFF transistors leak far less than one, because the middle node rises (about 0.17 V for our NAND2 '00'), giving the top device negative Vgs, body effect and less DIBL. Figure 7.2: NAND2 '00' leaks about 200× less than NAND2 '01'.

Q. Why is your leakage defined as power/VDD?

> Gate-tunnelling current flowing into an input pin is supplied, in a real circuit, by the previous gate from VDD. Counting all source power makes gate leakages add up to the circuit's. We also report plain I(VDD); the conclusions are the same.

Q. Why is the adder less accurate than the gates?

> The hierarchy assumes ideal 0/VDD levels and no loading, but real logic-0 levels are typically 6-21 mV (up to 100 mV at hot corners) at this node. Our decomposition (Figure 8.2) separates the two: the hierarchy alone costs a −2.6 % bias, while the ML part adds only about 1.3 % median error, and flat ML on the real node voltages is 1.2 %.

Q. What was the hardest bug?

> ngspice's altermod silently ignores all but the first parameter on a line. We caught it by comparing the fast simulation path against standalone netlists.

Q. You added extra data: is that allowed?

> Yes: the brief says the sample count may be increased. The deliverable CSVs are still the 10,000 uniform samples. The supplement targets the rail voltages where gates operate (distribution shift), and it used no gate or circuit simulations.

Q. Why is PMOS OFF gate current so poorly predicted?

> It is about 1e-15 A, at the level of double-precision rounding in SPICE itself. Those values are noise, which is why we exclude currents below 1e-14 A from percentage errors and report the fraction kept.

Q. Your transistor test error was below 1 %, but the gates were worse. How did you find out why?

> With a 'transistor oracle': re-simulate each transistor of a gate alone at its exact in-gate voltages and compare it with the ML, device by device. It showed a −27 % error for an OFF NMOS whose gate sat at 16 mV instead of 0 (the model had no Vg input) and a +3.6 % bias at narrow widths (effective width W − 2·wint not represented). Adding both as physics features fixed them.

Q. What is effective width and why does it matter?

> BSIM4 removes wint = 5 nm from each channel edge, so a drawn 16 nm device conducts like 6 nm. Current scales with W_eff, so we predict conductance per unit W_eff and give log(W_eff/L) as a feature.

Q. What would you improve?

> Solve internal node voltages with the ML models (no SPICE at all), propagate non-ideal logic levels in the hierarchy instead of assuming exact 0/VDD, and ensemble several networks to tighten the tails.

A Appendix

How to regenerate everything

```
python 01_generate_transistor_data.py   # 40,000 + 20,000 SPICE samples, ~2 min
python 02_data_check.py                 # Kirchhoff check, trend plots
python 03_train_models.py               # 4 families x 4 cases, ~101 min on 14 cores
python 04_gate_validation.py            # Part B
python 05_circuit_validation.py         # Part C
python 06_ablation_and_learning_curve.py   # extra analysis
python guide/build_guide.py             # this PDF
```

Every random draw comes from fixed seeds in `config.py` (data 2026, split 42, corners 7).

Where everything is

Deliverable (brief §6)	File(s)
1. Datasets (4 × 10,000 rows)	`data/nmos_on.csv … data/pmos_off.csv`
2. Example netlists	`netlists/single_*.cir, netlists/gate_*.cir, netlists/cla3.cir`
3. Data check	`results/kirchhoff_check.csv, figures/data_check_trends.png`
4. Model comparison	`results/model_comparison.csv (+ _all_hparams)`
5. Chosen models	`results/chosen_models_test.csv, figures/pred_vs_spice_*.png, saved_models/*.joblib`
6-7. Gate results and parity	`results/gate_results.csv, figures/gate_parity.png`
8. Test-circuit results	`results/circuit_results.csv, circuit_summary.csv, figures/circuit_parity.png`
9. Code	`01_ … 06_ scripts, config.py, leakml/`

Glossary

BSIM4	The industry-standard compact transistor model used by our model card (level 54).
Corner	One random draw of VDD, T, NMOS process and PMOS process parameters.
DIBL	Drain-induced barrier lowering: high V_{DS} lowers V_{th} in short channels.
GIDL	Gate-induced drain leakage: band-to-band tunnelling at the gate-drain overlap.
Bipole	A two-terminal current path inside the transistor (e.g. gate→drain tunnelling).
Passive	Current flows from higher to lower voltage, so it only absorbs power.
Distribution shift	Training data drawn from a different region than where the model is used.
L-BFGS	A quasi-Newton optimiser that uses curvature estimates; excellent for small, noise-free fitting.
p95	95th percentile: 95 % of errors are smaller than this.
Stack effect	Series OFF transistors leak much less than one, because the middle node rises.
Tellegen's theorem	In any circuit, power delivered by the sources = power absorbed by the elements.
V_T	Thermal voltage kT/q ≈ 26 mV at 300 K (not to be confused with V_{th}).